



File System Management

Managing file systems on Linux is a crucial task that involves organizing, storing, and maintaining data on a disk or other storage device. Linux is a versatile operating system that supports many different file systems, including ext2, ext3, ext4, XFS, Btrfs, and NTFS, among others. Each of these file systems has unique features and is suited to specific use cases. The best file system choice depends on the specific requirements of the application or user such as:

- **ext2** is an older file system with no journaling capabilities, which makes it less suited for modern systems but still useful in certain low-overhead scenarios (like USB drives).
- **ext3** and **ext4** are more advanced, with journaling (which helps in recovering from crashes), and ext4 is the default choice for most modern Linux systems because it offers a balance of performance, reliability, and large file support.
- **Btrfs** is known for advanced features like snapshotting and built-in data integrity checks, making it ideal for complex storage setups.
- **XFS** excels at handling large files and has high performance. It is best suited for environments with high I/O demands
- **NTFS**, originally developed for Windows, is useful for compatibility when dealing with dual-boot systems or external drives that need to work on both Linux and Windows systems.

When selecting a file system, it's essential to analyze the needs of the application or user factors such as performance, data integrity, compatibility, and storage requirements will influence the decision.

Linux's file system architecture is based on the Unix model, organized in a hierarchical structure. This structure consists of several components, the most critical being `inodes`. `Inodes` are data structures that store metadata about each file and directory, including permissions, ownership, size, and timestamps. Inodes do not store the file's actual data or name, but they contain pointers to the blocks where the file's data is stored on the disk.

The `inode` table is a collection of these inodes, essentially acting as a database that the Linux kernel uses to track every file and directory on the system. This structure allows the operating system to efficiently access and manage files. Understanding and managing inodes is a crucial aspect of file system management in Linux, especially in scenarios where a disk is running out of inode space before running out of actual storage capacity.

Let's use an analogy, think of the Linux file system like a library. The `inodes` are like index cards in the library's catalog system (`inode table`). Each card contains detailed information about a book (file) its title, author, location, and other details but not the actual book. The `inode` table is the entire catalog that helps the library (operating system) quickly find and manage the books (files).

In Linux, files can be stored in one of several key types:

- Regular files
- Directories
- Symbolic links

Regular Files

Regular files are the most common type and typically consist of text data (such as ASCII) and/or binary data (such as images, audio, or executables). They reside in various directories throughout the file system, not just in the root directory. The root directory (/) is simply the top of the hierarchical directory tree, and files can exist in any directory within that structure.

Directories

Directories are special types of files that act as containers for other files (both regular files and other directories). When a file is stored in a directory, that directory is referred to as the file's parent directory. Directories help organize files within the Linux file system, allowing for an efficient way to manage collections of files.

Symbolic Links

In addition to regular files and directories, Linux also supports symbolic links (`symlinks`), which act as shortcuts or references to other files or directories. Symbolic links allow quick access to files located in different parts of the file system without duplicating the file itself. Symlinks can be used to streamline access or organize complex directory structures by pointing to important files across various locations.

Each category of user can have different permission levels. For example, the owner of a file may have permission to read, write, and execute it, while others may only have read access. These permissions are independent for each category, meaning changes to one user's permissions do not necessarily affect others.

shellsession

```
athane@htb[/htb]$ ls -il
```

```
total 0
```

```
10678872 -rw-r--r--  1 cry011t3  htb  234123 Feb 14 19:30 myscript.py
```

```
10678869 -rw-r--r--  1 cry011t3  htb   43230 Feb 14 11:52 notes.txt
```

Disks & Drives

Disk management on Linux involves managing physical storage devices, including hard drives, solid-state drives, and removable storage devices. The main tool for disk management on Linux is the `fdisk`, which allows us to create, delete, and manage partitions on a drive. It can also display information about the partition table, including the size and type of each partition. Partitioning a drive on Linux involves dividing the physical storage space into separate, logical sections. Each partition can then be formatted with a specific file system, such as ext4, NTFS, or FAT32, and can be mounted as a separate file system. The most common partitioning tool on Linux is also `fdisk`, `gpart`, and `GParted`.

Fdisk

```
athane@htb[/htb]$ sudo fdisk -l
```

```
Disk /dev/vda: 160 GiB, 171798691840 bytes, 335544320 sectors
```

```
Units: sectors of 1 * 512 = 512 bytes
```

```
Sector size (logical/physical): 512 bytes / 512 bytes
```

```
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

```
Disklabel type: dos
```

shellsession

```
Disk identifier: 0x5223435f
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/vda1	*	2048	158974027	158971980	75.8G	83	Linux
/dev/vda2		158974028	167766794	8792767	4.2G	82	Linux swap / Solaris

```
Disk /dev/vdb: 452 KiB, 462848 bytes, 904 sectors
```

```
Units: sectors of 1 * 512 = 512 bytes
```

```
Sector size (logical/physical): 512 bytes / 512 bytes
```

```
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

Mounting

Each logical partition or storage drive must be assigned to a specific directory in the file system. This process is known as **mounting**. Mounting involves linking a drive or partition to a directory, making its contents accessible within the overall file system hierarchy. Once a drive is mounted to a directory (also called a mount point), it can be accessed and used like any other directory on the system.

The **mount** command is commonly used to manually mount file systems on Linux. However, if you want certain file systems or partitions to be automatically mounted when the system boots, you can define them in the **/etc/fstab** file. This file lists the file systems and their associated mount points, along with options like read/write permissions and file system types, ensuring that specific drives or partitions are available upon startup without needing manual intervention.

Mounted File systems at Boot

shellsession

```
athane@htb[/htb]$ cat /etc/fstab

# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a device; this may
# be used with UUID= as a more robust way to name devices that works even if
# disks are added and removed. See fstab(5).
#
# <file system>          <mount point> <type>  <options>  <dump>  <pass>
UUID=3d6a020d-...SNIP...-9e085e9c927a /              btrfs
subvol=@,defaults,noatime,nodiratime,nodatacow,space_cache,autodefrag 0 1
UUID=3d6a020d-...SNIP...-9e085e9c927a /home           btrfs
subvol=@home,defaults,noatime,nodiratime,nodatacow,space_cache,autodefrag 0 2
UUID=21f7eb94-...SNIP...-d4f58f94e141 swap             swap      defaults,noatime 0 0
```

To view the currently mounted file systems, we can use the `mount` command without any arguments. The output will show a list of all the currently mounted file systems, including the device name, file system type, mount point, and options.

List Mounted Drives

shellsession

```
athane@htb[/htb]$ mount

sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs
(rw,nosuid,relatime,size=4035812k,nr_inodes=1008953,mode=755,inode64)
```

```
devpts on /dev/pts type devpts
(rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
tmpfs on /run type tmpfs
(rw,nosuid,nodev,noexec,relatime,size=814580k,mode=755,inode64)
/dev/vda1 on / type btrfs
(rw,noatime,nodiratime,nodatasum,nodatacow,space_cache,autodefrag,subvolid=257,subvol=
```

To mount a file system, we can use the `mount` command followed by the device name and the mount point. For example, to mount a USB drive with the device name `/dev/sdb1` to the directory `/mnt/usb`, we would use the following command:

Mount a USB drive

```
athane@htb[/htb]$ sudo mount /dev/sdb1 /mnt/usb
athane@htb[/htb]$ cd /mnt/usb && ls -l
```

shellsession

```
total 32
drwxr-xr-x 1 root root 18 Oct 14 2021 'Account Takeover'
drwxr-xr-x 1 root root 18 Oct 14 2021 'API Key Leaks'
drwxr-xr-x 1 root root 18 Oct 14 2021 'AWS Amazon Bucket S3'
drwxr-xr-x 1 root root 34 Oct 14 2021 'Command Injection'
drwxr-xr-x 1 root root 18 Oct 14 2021 'CORS Misconfiguration'
drwxr-xr-x 1 root root 52 Oct 14 2021 'CRLF Injection'
drwxr-xr-x 1 root root 30 Oct 14 2021 'CSRF Injection'
drwxr-xr-x 1 root root 18 Oct 14 2021 'CSV Injection'
drwxr-xr-x 1 root root 1166 Oct 14 2021 'CVE Exploits'
...SNIP...
```

To unmount a file system in Linux, we can use the `umount` command followed by the mount point of the file system we want to unmount. The mount point is the location in the file system where the file system is mounted and is accessible to us. For example, to unmount the USB drive that was previously mounted to the directory `/mnt/usb`, we would use the following command:

Unmount

```
athane@htb[/htb]$ sudo umount /mnt/usb
```

shellsession

It is important to note that we must have sufficient permissions to unmount a file system. We also cannot unmount a file system that is in use by a running process. To ensure that there are no running processes that are using the file system, we can use the `lsof` command to list the open files on the file system.

```
cry011t3@htb:~$ lsof | grep cry011t3
```

shellsession

```
vncserver 6006      cry011t3  mem      REG      0,24      402274 /usr/bin/perl  
(path dev=0,26)  
vncserver 6006      cry011t3  mem      REG      0,24      1554101  
/usr/lib/locale/aa_DJ.utf8/LC_COLLATE (path dev=0,26)  
vncserver 6006      cry011t3  mem      REG      0,24      402326 /usr/lib/x86_64-  
linux-gnu/perl-base/auto/POSIX/POSIX.so (path dev=0,26)  
vncserver 6006      cry011t3  mem      REG      0,24      402059 /usr/lib/x86_64-  
linux-gnu/perl/5.32.1/auto/Time/HiRes/HiRes.so (path dev=0,26)
```



```
vncserver 6006      cry011t3  mem      REG      0,24      1444250 /usr/lib/x86_64-  
linux-gnu/libnss_files-2.31.so (path dev=0,26)  
vncserver 6006      cry011t3  mem      REG      0,24      402327 /usr/lib/x86_64-  
linux-gnu/perl-base/auto/Socket/Socket.so (path dev=0,26)  
vncserver 6006      cry011t3  mem      REG      0,24      402324 /usr/lib/x86_64-  
linux-gnu/perl-base/auto/IO/IO.so (path dev=0,26)  
...SNIP...
```

If we find any processes that are using the file system, we need to stop them before we can unmount the file system. The `/etc/fstab` file contains information about file systems that should be mounted on the system, including options for automatic mounting at boot time and other mount options. All mounted filesystems (whether listed in `/etc/fstab` or mounted manually) will be cleanly unmounted automatically during system shutdown by the OS. If we want to prevent a filesystem from mounting automatically at boot, we need to add the `noauto` option to its entry in the `/etc/fstab` file. This would look like, for example, the following:

Fstab File

```
/dev/sda1 / ext4 defaults 0 0  
/dev/sda2 /home ext4 defaults 0 0  
/dev/sdb1 /mnt/usb ext4 rw,noauto,user 0 0  
192.168.1.100:/nfs /mnt/nfs nfs defaults 0 0
```

txt

SWAP

Swap space is an essential part of memory management in Linux and plays a critical role in ensuring smooth system performance, especially when the available physical memory (RAM) is fully utilized. When the system runs out of physical memory, the kernel moves inactive pages of memory (data not immediately in use) to the swap space, freeing up RAM for active processes. This process is known as swapping.

Creating Swap Space

Swap space can be set up either during the installation of the operating system or added later using the `mkswap` and `swapon` commands.

- `mkswap` is used to prepare a device or file to be used as swap space by creating a Linux swap area
- `swapon` activates the swap space, allowing the system to use it

Sizing and Managing Swap Space

The size of the `swap space` is not fixed and depends on your system's physical memory and intended usage. For example, a system with less RAM or running memory-intensive applications might need more swap space. However, modern systems with large amounts of RAM may require less or even no swap space, depending on specific use cases.

When setting up swap space, it's important to allocate it on a dedicated partition or file, separate from the rest of the file system. This prevents fragmentation and ensures efficient use of the swap area when needed. Additionally, because sensitive data can be temporarily stored in swap space, it's recommended to encrypt the swap space to safeguard against potential data exposure.

Swap Space for Hibernation

Besides extending physical memory, swap space is also used for **hibernation**. Hibernation is a power-saving feature that saves the system's state (including open applications and processes) to the swap space and powers off the system. When the system is powered back on, it restores its previous state from the swap space, resuming exactly where it left off.

Connect to HTB



Enable step-by-step solutions ⓘ PRO

Question 1

XP +20 ^

How many partitions exist in our Pwnbox? (Format: 0)

Write your answer

Submit



22 / 30 Sections



